

Diffusion Models Beat GANs on Image Synthesis (OpenAI, NIPS 2021)

이미지 처리팀

김수용, 김태수, 김현진, 안가영, 장보아, 최승준, 허정원

Content

- Introduction
- Background
- Architecture Improvements
- Adaptive Group Normalization (AdaGN)
- Classifier Guidance
- Results
- Limitations and Future Work

Introduction

- 그동안의 Diffusion Model



DDPM
2020년 6월



Stable Diffusion
2022년 6월



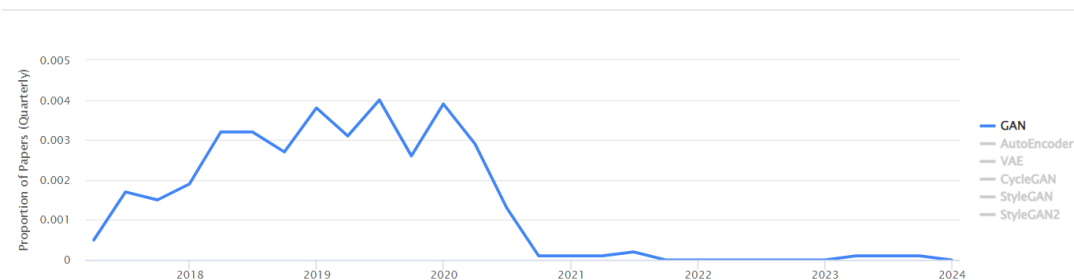
SORA
2024년 2월

Introduction

전체 딥러닝 관련 논문 중 비율 (Papers with code 논문 찾기 사이트)

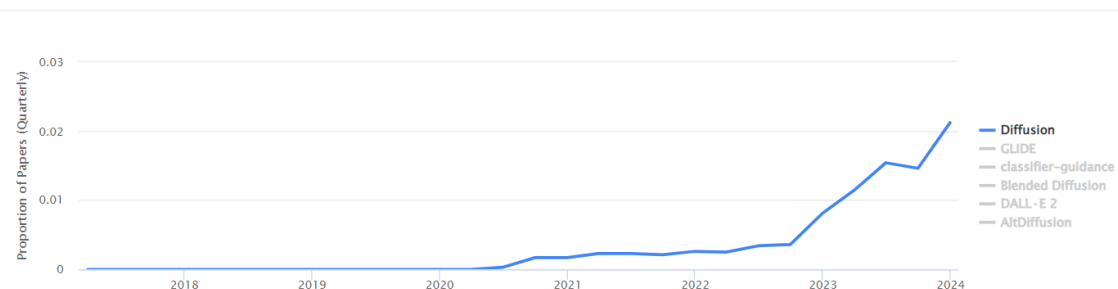
GAN

Usage Over Time



Diffusion

Usage Over Time



- GAN (Generative Adversarial Network)은 2014년에 나온 이후, 2021년까지 이미지 생성에 SOTA를 달하는 방식이었음.
- 그동안 Diffusion 기반의 모델들이 발전했지만, 여전히 GAN의 성능을 따라잡지는 못했음.
- 이번 논문(Ablated)에서 GAN을 제치고 대부분의 이미지 생성 데이터셋에서 SOTA가 되었다. -> 2021년

Introduction

Diffusion Model의 성능 문제와 이번 논문의 해결방법

1. GAN 논문이 그동안 많이 나와서 Diffusion 모델보다 모델 아키텍처가 더 탐구되고 정제되었다.

→ **U-Net 기반의 Diffusion 모델의 세부적인 아키텍처를 개선함.**

이미지 생성 모델이 좋은지 평가하는 방법은? Fidelity vs Diversity

Fidelity: 생성된 이미지와 실제 이미지가 얼마나 유사한지

Diversity: 생성된 이미지가 얼마나 다양하게 이미지를 생성하는지

2. GAN은 fidelity와 diversity의 trade-off (반비례 관계)에서 fidelity를 선택하였다

→ **Classifier Guidance를 이용!**

→ **Fidelity를 위해 Diversity를 희생하는 방법을 고안.**

→ **Class를 Condition으로 입력 받아 목표한 Class 이미지를 생성할 수 있다.**

- 이러한 개선 사항을 통해 여러 가지 metric과 데이터셋에서 GAN을 능가하는 새로운 모델을 만들었다.

Background

- **Diffusion Model : 확산 모델**
- 확산(Diffusion) 현상에 대해 하나의 분자에만 초점을 맞춰서 보면 분자 하나는 가우시안 분포를 따름.

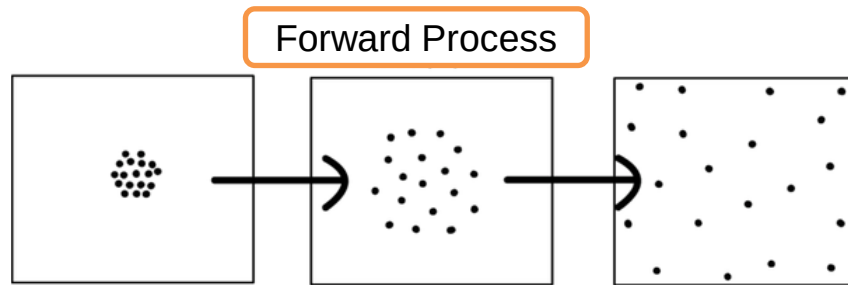


그림4. Forward Diffusion Process

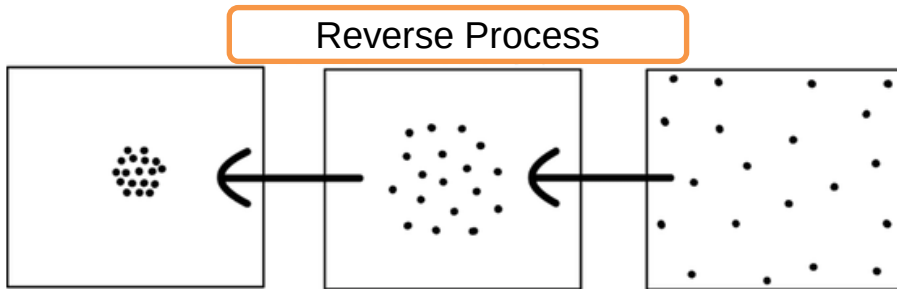


그림6. Reverse Diffusion Process

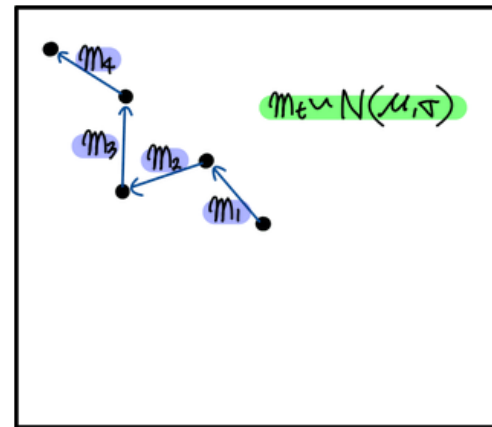
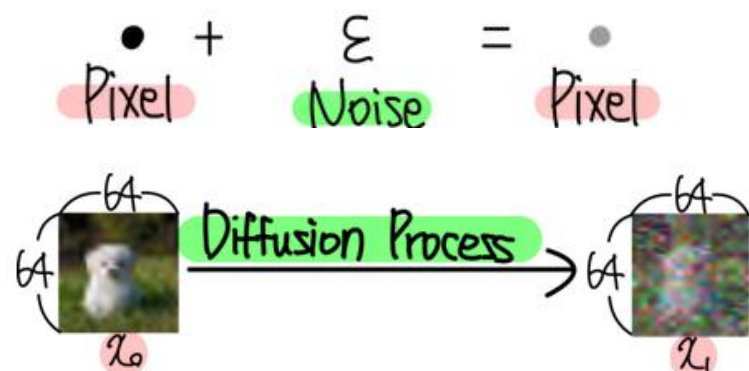
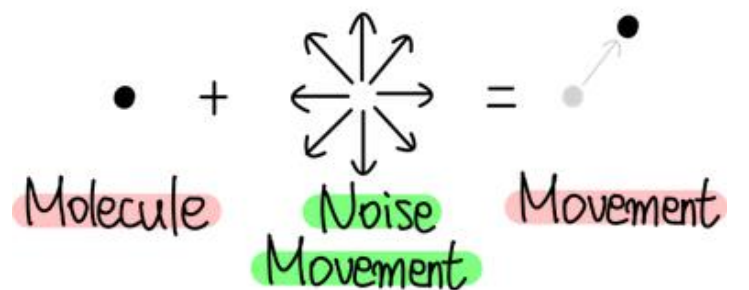


그림5. 분자 하나의 움직임

Background

- 분자의 움직임을 이미지의 pixel로 전환하면, 임의의 노이즈가 더해졌다고 생각할 수 있음.
- 64 x 64 의 이미지가 있다면, 64 x 64 개의 분자가 돌아다닌다고 봐도 됨.



$$x_{1,i} = x_{0,i} + \epsilon_{0,i} \quad (1 \leq i \leq 64 \times 64)$$

$$\epsilon \sim N(\mu, \sigma)$$

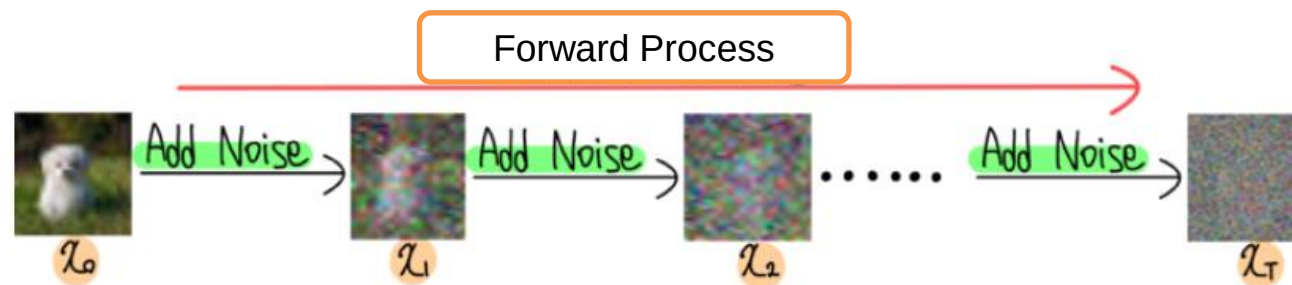


그림9. 이미지의 Forward Diffusion Process

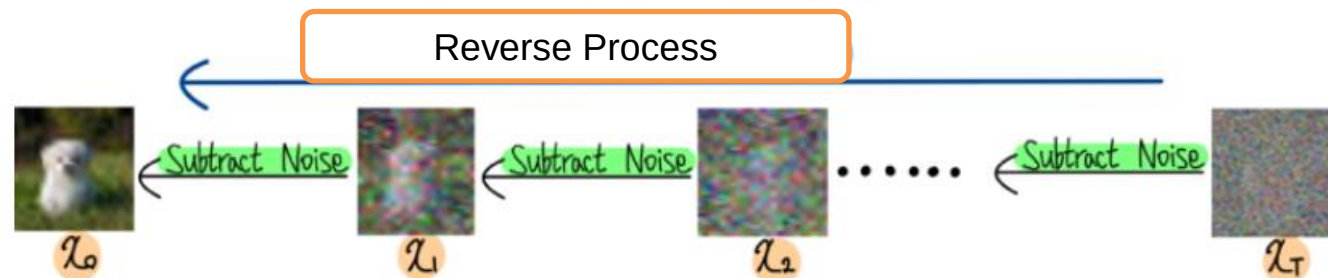


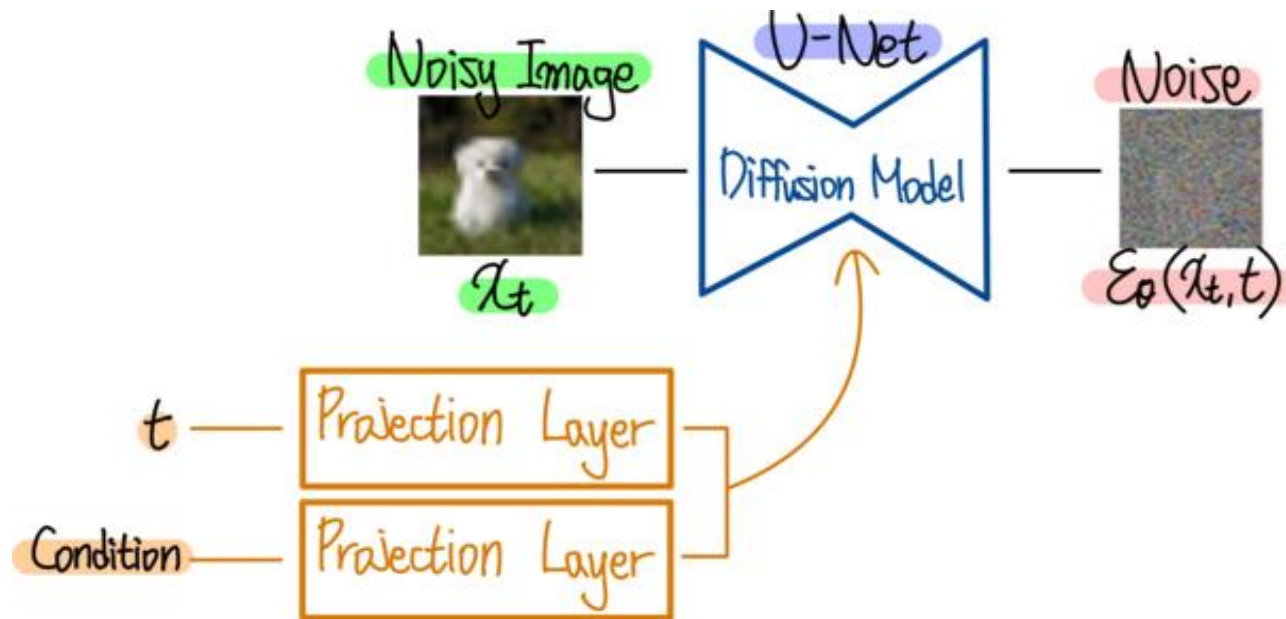
그림10. 이미지의 Reverse Diffusion Process

Background

- Diffusion Model은 noisy image를 Input으로 받아, 현재 시점의 추가된 noise를 예측한다.

- Diffusion 모델 구성요소
 1. Input 이미지와 Noisy 이미지
 2. 몇 번째 Process 인지를 나타내는 time step t
 3. Condition (조건): Text, Mask 등
 4. U-Net 기반의 노이즈를 추정하는 Diffusion Model
 5. Output (Input과 차원수가 같음)

- Loss Function
- 실제 Noise와 모델이 예측한 Noise 사이의 Mean Squared Loss (Simple Loss)



$$L_{\text{simple}}(\theta) := \mathbb{E}_{t, x_0, \epsilon} \left[\left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2 \right]$$

실제 Noise
모델이 예측한 Noise

그림12. Diffusion Model Loss Function

Background

- Diffusion Model은 noisy image를 Input으로 받아, **현재 시점의 추가된 noise**를 예측한다.

- Diffusion 모델 구성요소

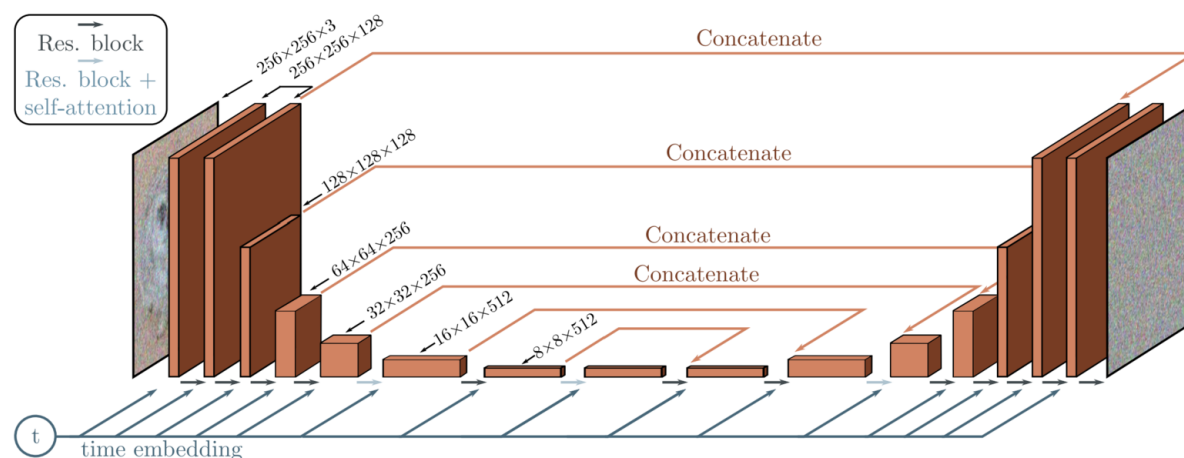
-
-
-
-

- 4. U-Net 기반의 노이즈를 추정하는 Diffusion Model

- 5. Output (Input과 차원수가 같음)

- Loss Function

- 실제 Noise와 모델이 예측한 Noise 사이의
Mean Squared Loss (Simple Loss)



실제 Noise

$$L_{\text{simple}}(\theta) := \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \left[\left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2 \right]$$

모델이 예측한 Noise

그림12. Diffusion Model Loss Function

Background

DDPM

- 기존 diffusion model의 loss term과 parameter estimation의 과정을 개선시킴
- forward process의 parameter를 미리 **정해진 상수**로 정의하여 별도의 학습이 필요하지 않게 함
- Reverse process의 손실함수를 간단하게 정의하여 전체 프로세스를 단순화시킴.

DDIM

- **Non-Markovian Process**를 도입해서 Sampling Speed를 높임.
- 이미지 퀄리티를 조금 희생하면서, 이미지 샘플링 속도를 향상시킴.

Improved DDPM

- Diffusion process 초기 몇 step이 성능에 대부분을 기여하기 때문에 **분산을 잘 선택**하면, 적은 step으로 높은 이미지 품질을 얻을 수 있음.
- DDPM에서 분산을 **상수**로 두는데, 그것이 아니라 **분산을 parametrize** 하는 것을 제안.

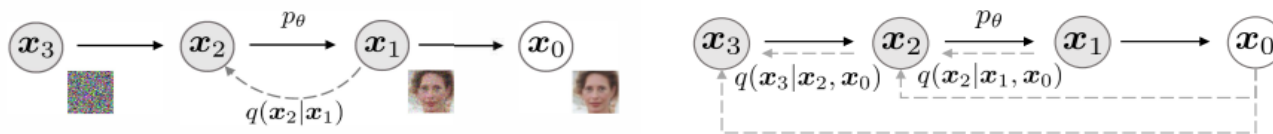


Figure 1: Graphical models for diffusion (left) and non-Markovian (right) inference models.

Background

- **Sample Quality Metrics**

- 실제 사용하는 metric(IS, FID, sFID)의 경우 사람들이 판단하는 정도와 종종 일치하지만, 완벽하지는 않다.

- **Inception Score (IS)**

- Sharpness x Diversity
- Variation이 클수록 Score가 크도록 설계됨.
- 하나의 클래스에 대한 다양성을 포착하는 데에는 적절하지 않다.
- 전체 데이터셋의 부분집합을 암기해서 생성하는 모델은 여전히 높은 IS를 갖게 된다.

Sharpness

$$S = \exp(E_{x \sim p}[\int c(y|x) \log c(y|x) dy])$$

$$IS = D \times S$$

Diversity

$$D = \exp(-E_{x \sim p}[\int c(y|x) \log c(y) dy])$$



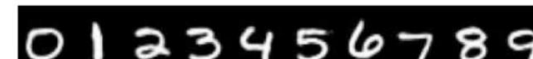
Low sharpness



High sharpness



Low diversity



High diversity

Classifier가 class를 더 잘 예측하는 이미지를 생성

최대한 다양한 class를 예측

Background

- **Sample Quality Metrics**

- 실제 사용하는 metric(IS, FID, sFID)의 경우 사람들이 판단하는 정도와 종종 일치하지만, 완벽하지는 않다.

- **Frechet Inception Distance (FID)**

- IS보다 diversity(하나의 클래스에 대해서도)를 포착하기 더 낫다.
- Inception V3라는 Image classification 모델을 이용해 latent space를 구하고, 데이터셋과 생성한 이미지의 latent space를 비교
- FID를 default metric으로 선정 - diversity,와 fidelity를 모두 포착함.
- Precision과 IS는 fidelity를 측정하기 위해, Recall은 diversity를 측정하기 위해.

$$\|m - m_w\|_2^2 + \text{Tr}(C + C_w - 2(CC_w)^{1/2})$$

m: 데이터셋의 latent vector 평균

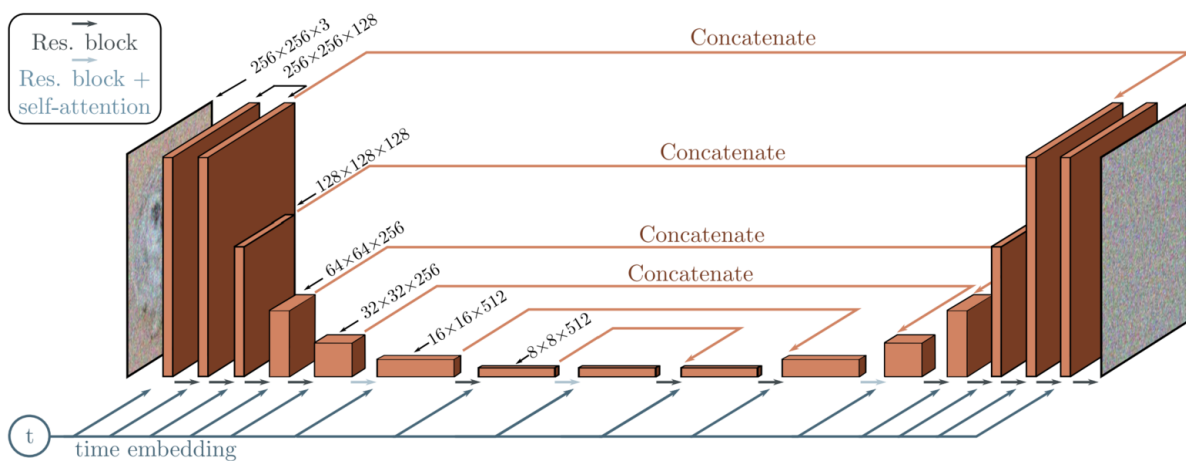
m_w: 생성된 latent vector 평균

C: 데이터셋의 latent vector 공분산

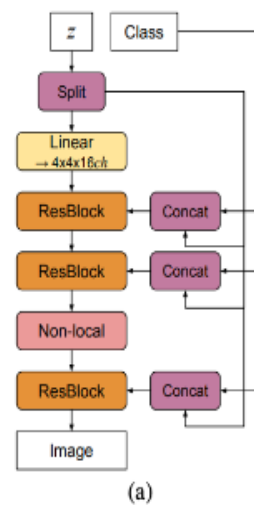
C_w: 생성된 latent vector 공분산

Background

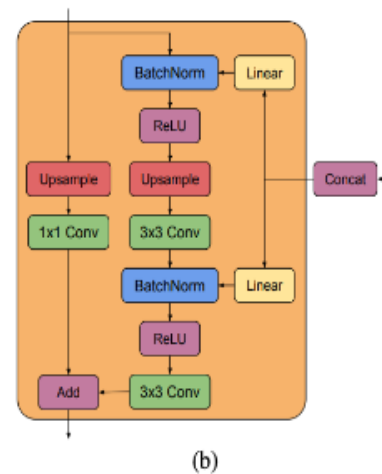
- **BigGAN (2018)**
- 이미지 생성에서 SOTA 성능을 가진 GAN 대표적인 모델
- 이름에서도 Big을 쓰는 만큼 나타내는 것처럼 기존 GAN의 파라미터의 2~4배의 파라미터를 가지고 있으며 batchsize를 8배 이상 키운 것이 특징
- orthogonal regularization과 함께 “**truncation trick**”을 사용해 결과 이미지의 fidelity와 variety 사이 trade-off 조절이 가능
- 이번 논문에서 BigGAN의 Residual Block을 사용해서 DDPM Unet구조에서 쓰이는 Residual block 대체.



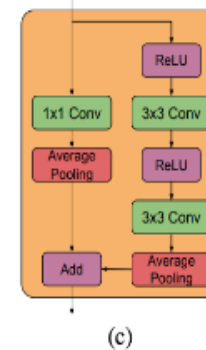
DDPM 모델 아키텍처



BigGAN의 G 의 대표적인 구조



BigGAN에 사용되는 Residual Block(ResBlockup)
(b): Generator
(c): Discriminator



Architecture Improvements

Diffusion 모델에 최적의 샘플 품질을 제공하는 모델 아키텍처를 찾기 위해 여러 아키텍처 변형을 수행.

- 모델의 크기를 일정하게 유지하면서 깊이/너비 증가
- **Attention heads의 수 증가**
- Attention 을 16×16 뿐 아니라 32×32, 16×16, and 8×8 resolution에서도 사용
- **BigGAN 의 residual block 을 upsampling 과 downsampling 에 사용**
- **Residual connection 을 $\frac{1}{\sqrt{2}}$ 로 rescale**

Channels	Depth	Heads	Attention resolutions	BigGAN up/downsample	Rescale resblock	FID 700K	FID 1200K
160	2	1	16	✗	✗	15.33	13.21
128	4	4	32,16,8	✓	✓	-0.21	-0.48
						-0.54	-0.82
						-0.72	-0.66
						-1.20	-1.21
160	2	4	32,16,8	✓	✗	0.16	0.25
						-3.14	-3.00

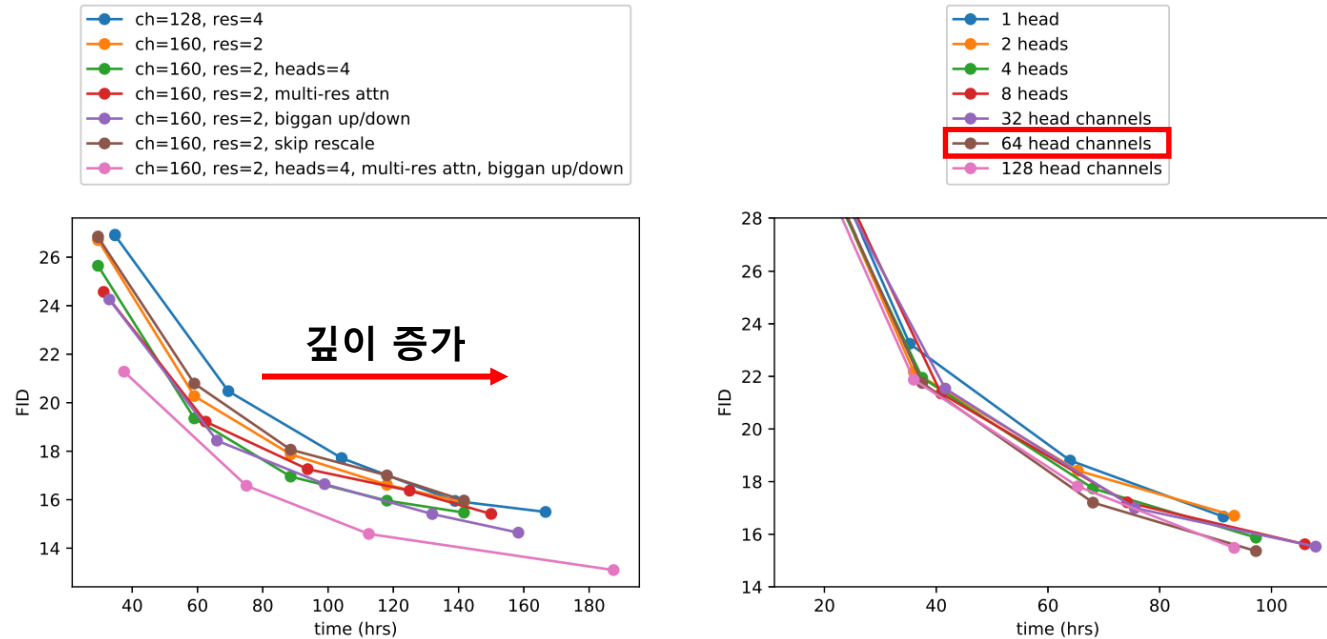
- Residual connection을 rescale하는 것을 제외하고는 모두 성능이 개선, 같이 사용하였을 때 효과.

Architecture Improvements

Number of heads	Channels per head	FID
1		14.08
2		-0.50
4		-0.97
8		-1.17
	32	-1.36
	64	-1.03
	128	-1.08

- Transformer 아키텍처와 더 잘 일치하는 다른 attention 구성을 연구.
- **헤드가 많을수록 또는 헤드별 채널수가 작을수록 FID가 개선됨.**

Architecture Improvements



- (좌측) **깊이 증가**가 성능에 도움이 되지만, 학습 시간이 늘어남. 더 넓은 모델과 같은 성능에 도달하는 데 더 오랜 시간이 걸림.
- (우측) **64 헤드별 채널수**가 실행시간 대비 가장 좋다는 것을 확인, 기본값으로 헤드당 64 채널을 사용.

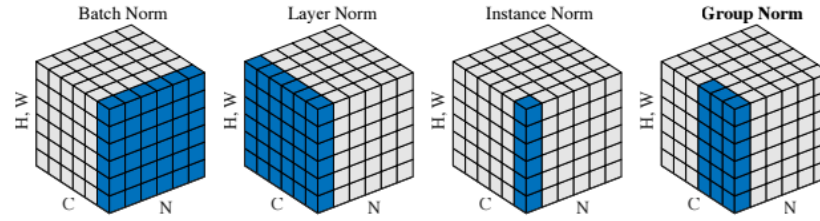
Architecture Improvements

Default Architecture

- 논문에서는 아래의 모델 아키텍처를 기본 값으로 사용
1. Resolution당 2개의 residual block
 2. 64개의 헤드별 채널
 3. (32, 16, 8) resolution서의 attention
 4. BIGGAN의 residual block으로 up/downsampling
 5. **Residual block에 timestep embedding과 class embedding을 주입하기 위한 AdaGN**

Adaptive Group Normalization (AdaGN)

Group Normalization



- 입력 특성을 그룹으로 나누고, 각 그룹 내에서 데이터의 평균과 분산을 계산하여 정규화 (**Layer Norm과 Instance Norm의 중간**)
- 배치 크기에 의존하지 않으며, 특히 작은 배치 크기에서 배치 정규화(Batch Normalization)보다 우수한 성능을 보이는 경우가 많음.

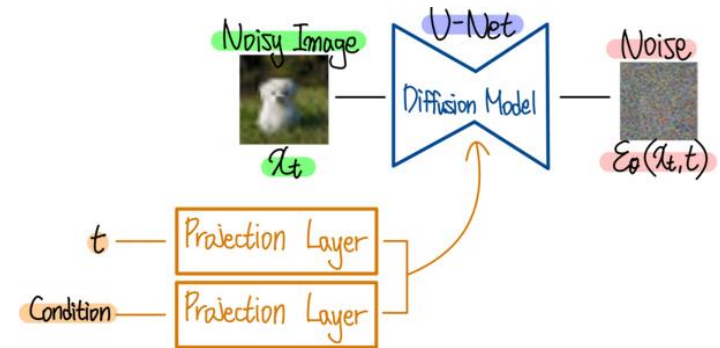
AdaGN

- AdaGN이라는 Layer로 실험을 진행, 이 layer 는 group normalization 연산 후, 각 residual block 에 timestep embedding과 class embedding을 결합함.

$$AdaGN(h, y) = y_s GroupNorm(h) + y_b$$

- h 는 residual block 중간의 activation
- $y=[y_s, y_b]$ 는 time step embedding (t) 과 class embedding (y)의 linear projection으로 구함
- 실제로 AdaGN을 이용하면 FID가 개선됨.

Operation	FID
AdaGN	13.06
Addition + GroupNorm	15.08



Q&A

?

Classifier Guidance

- **Conditional Reverse Noising Process**

- 잘 설계된 아키텍처를 사용하는 것 외에도 조건부 이미지 합성을 위한 GAN은 클래스 레이블을 많이 사용
- Diffusion 모델에서도 클래스 정보가 이미지를 생성하는 데에 도움이 됨.
- 일단 AdaGN에서 클래스 정보(class embedding y_b)를 AdaGN으로 주입하고 있다.
- 여기에서 **diffusion generator**를 개선하기 위해 **Classifier**를 활용하는 다른 접근 방식이 필요하다.
- Noisy한 이미지 x_t 를 classifier에 학습시킨다. Classifier의 Freeze하고, 그 gradient를 Diffusion model에 사용한다.

ex) 강아지 이미지를 생성하려고 할 때, Diffusion model이 고양이를 생성하려고 하면, classifier에서 큰 gradient를 줘서 강아지를 생성하도록 반영.

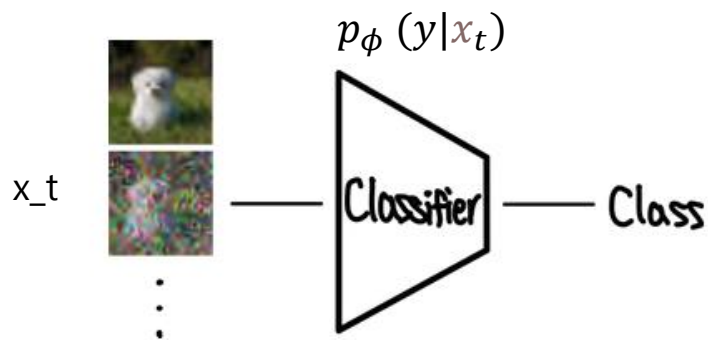
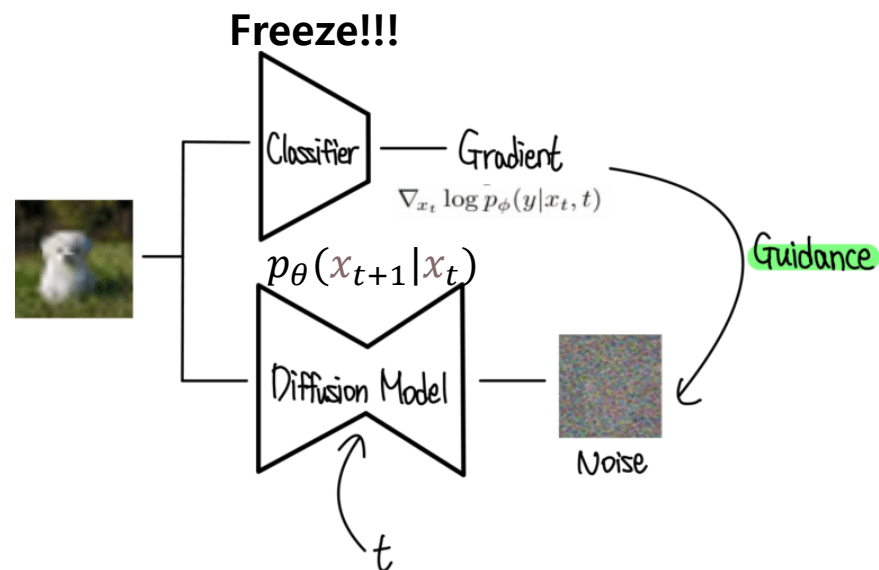


그림2. Noisy Image Classifier



Classifier Guidance

- **Conditional Reverse Noising Process**

- 학습된 Classifier를 어떻게 활용하는지 보면,
- p_θ : reverse process
- p_ϕ : classifier
- Z : 확률 분포 합 1로 만드는 상수

- **Reverse Process x Classifier**

$$p_{\theta,\phi}(x_t|x_{t+1}, y) = Zp_\theta(x_t|x_{t+1})p_\phi(y|x_t)$$

- Reverse process 분포는 다음과 같이 전개 가능.

$$p_\theta(x_t|x_{t+1}) = \mathcal{N}(\mu, \Sigma)$$

$$\log p_\theta(x_t|x_{t+1}) = -\frac{1}{2}(x_t - \mu)^T \Sigma^{-1}(x_t - \mu) + C$$

Classifier Guidance

- **Conditional Reverse Noising Process**

- Diffusion Step이 무한대의 step으로 가면 분산 자체가 0으로 감. (주변 변화량이 적음) → $\log_\phi p(y|x_t)$ 평평한 분포라고 가정.
- 따라서 $x_t = \mu$ 에 대해 Taylor expansion으로 근사하면 이것을 다른 x_t 영역에도 적용할 수 있다.

$$\log p_\phi(y|x_t) \approx \log p_\phi(y|x_t)|_{x_t=\mu} + (x_t - \mu)^T \nabla_{x_t} \log p_\phi(y|x_t)|_{x_t=\mu}$$

- $$= (x_t - \mu)^T g + C_1$$
$$g = \nabla_{x_t} \log p_\phi(y|x_t)|_{x_t=\mu},$$

$$\begin{aligned} \log(p_\theta(x_t|x_{t+1})p_\phi(y|x_t)) &\approx -\frac{1}{2}(x_t - \mu)^T \Sigma^{-1}(x_t - \mu) + (x_t - \mu)^T g + C_2 \\ &= -\frac{1}{2}(x_t - \mu - \Sigma g)^T \Sigma^{-1}(x_t - \mu - \Sigma g) + \frac{1}{2}g^T \Sigma g + C_2 \\ &= -\frac{1}{2}(x_t - \mu - \Sigma g)^T \Sigma^{-1}(x_t - \mu - \Sigma g) + C_3 \\ &= \log p(z) + C_4, z \sim \mathcal{N}(\mu + \Sigma g, \Sigma) \end{aligned}$$

- 전개하면 학습된 classifier에 의해 분산 x gradient 로 평균이 shifted된 분포를 가짐.

Classifier Guidance

- **Conditional Reverse Noising Process**

Algorithm 1 Classifier guided diffusion sampling, given a diffusion model $(\mu_\theta(x_t), \Sigma_\theta(x_t))$, classifier $p_\phi(y|x_t)$, and gradient scale s .

Input: class label y , gradient scale s

$x_T \leftarrow \text{sample from } \mathcal{N}(0, \mathbf{I})$

for all t from T to 1 **do**

$\mu, \Sigma \leftarrow \mu_\theta(x_t), \Sigma_\theta(x_t)$

$x_{t-1} \leftarrow \text{sample from } \mathcal{N}(\mu + s\Sigma \nabla_{x_t} \log p_\phi(y|x_t), \Sigma)$

end for

return x_0

1. 정규분포에서 x_t 추출 (노이즈 이미지)
2. t 시점에서 x_t 의 평균과 분산을 구함.
3. (평균 + scaling factor x 분산 x classifier gradient, 분산)으로 이루어진 gaussian distribution에 샘플링하기
4. t 가 T (노이즈)에서 1(이미지)가 될 때까지 계속 반복.

Classifier Guidance

- **Conditional Sampling for DDIM**

- Conditional Sampling은 Stochastic Diffusion sampling에서만 적용 가능. DDIM 모델에서는 불가.

- $p_\theta(x_t)$ (prior)의 score function은 다음과 같이 정의가 된다.

$$\nabla_{x_t} \log p_\theta(x_t) = -\frac{1}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t)$$

$$\alpha_t := 1 - \beta_t, \quad \bar{\alpha}_t := \prod_{s=1}^t \alpha_s$$

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$$

$$q(x_{1:T}|x_0) := \prod_{t=1}^T q(x_t|x_{t-1})$$

$$q(x_t|x_{t-1}) := \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$

- 위 식에서 class condition을 넣어 $p_\theta(x_t)p_\phi(y|x_t)$ 의 score function에 대입하면,

$$\nabla_{x_t} \log(p_\theta(x_t)p_\phi(y|x_t)) = \nabla_{x_t} \log p_\theta(x_t) + \nabla_{x_t} \log p_\phi(y|x_t)$$

$$-\frac{1}{\sqrt{1 - \bar{\alpha}_t}} \hat{\epsilon}(x_t) = -\frac{1}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t) + \nabla_{x_t} \log p_\phi(y|x_t)$$

- 새로운 noise 예측 모델을 다음과 같이 정의할 수 있음.

$$\hat{\epsilon}(x_t) := \epsilon_\theta(x_t) - \sqrt{1 - \bar{\alpha}_t} \nabla_{x_t} \log p_\phi(y|x_t)$$

Classifier Guidance

- Conditional Sampling for DDIM

Algorithm 2 Classifier guided DDIM sampling, given a diffusion model $\epsilon_\theta(x_t)$, classifier $p_\phi(y|x_t)$, and gradient scale s .

Input: class label y , gradient scale s

$x_T \leftarrow$ sample from $\mathcal{N}(0, \mathbf{I})$

for all t from T to 1 **do**

$\hat{\epsilon} \leftarrow \epsilon_\theta(x_t) - \sqrt{1 - \bar{\alpha}_t} \nabla_{x_t} \log p_\phi(y|x_t)$

$x_{t-1} \leftarrow \sqrt{\bar{\alpha}_{t-1}} \left(\frac{x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}}{\sqrt{\bar{\alpha}_t}} \right) + \sqrt{1 - \bar{\alpha}_{t-1}} \hat{\epsilon}$

end for

return x_0

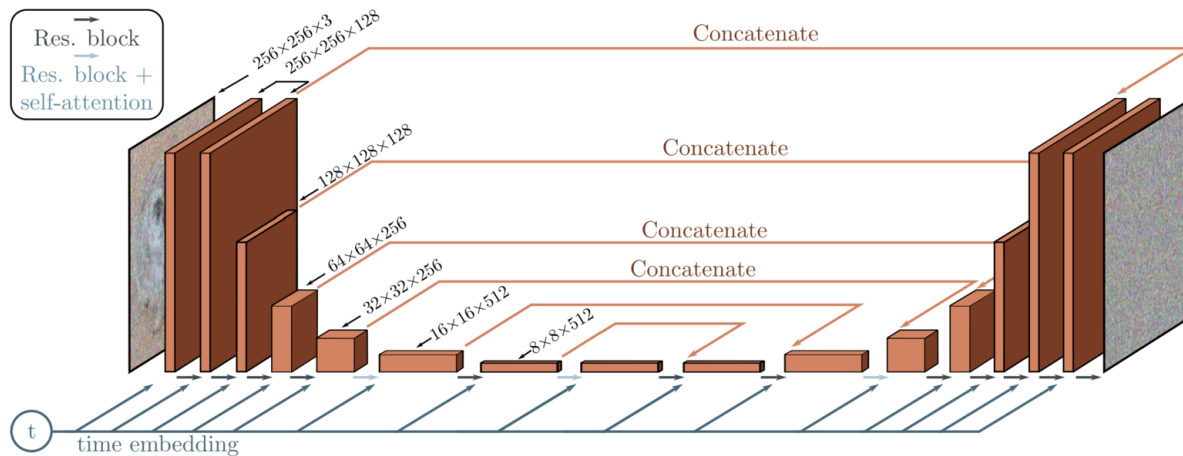
1. 정규분포에서 x_t 추출 (노이즈 이미지)
2. t 시점에서 T 에서 1로 갈수록(노이즈를 제거하는 방향) 입실론에 classifier 반영하고 반영한 입실론을 이용해서 x_{t-1} 를 구함.
3. t 가 T (노이즈)에서 1(이미지)가 될 때까지 계속 반복.

Classifier Guidance

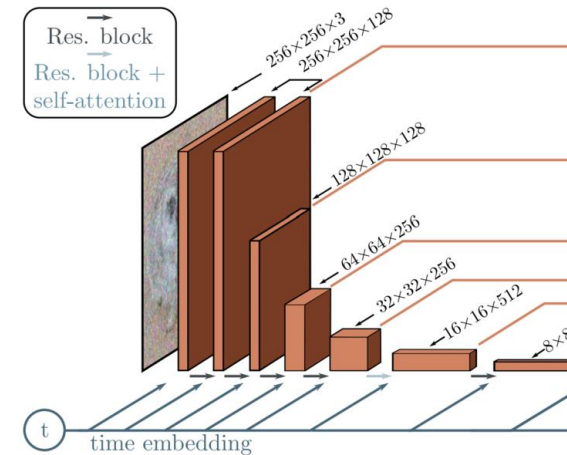
Scaling Classifier Gradients

- 대규모 생성 task에 classifier guidance를 적용하기 위해 imagenet에서 classification 모델을 학습시킴.
- Classifier는 UNet 모델의 downsampling 부분을 사용하고 Output을 위해 8x8 layer에 attention pool이 있음. Diffusion model과 동일한 노이즈 분포에서 classifier를 학습시킴.

전체 ADM 아키텍처



Classifier



Attention Pooling
=
Attention+ Global
Average Pooling

Classifier Guidance

Scaling Classifier Gradients

- **Scale로 1**을 사용하면 classifier가 합리적인 확률(약 50%)을 원하는 클래스에 할당한다. 그러나 생성된 이미지가 의도한 클래스와 일치하지 않는다. Classifier gradient를 **1보다 훨씬 크게 하면** classifier의 클래스 확률이 거의 100%가 됨.



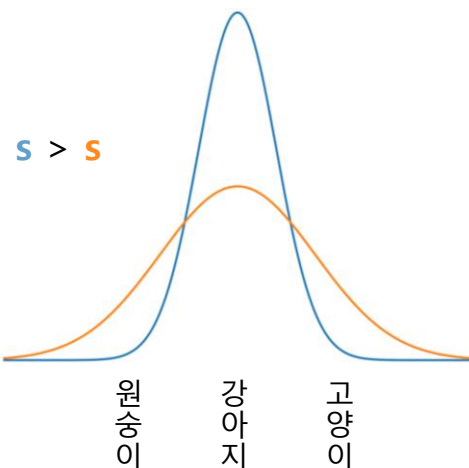
- Classifier guidance를 사용한 unconditional diffusion model (클래스를 따로 지정하지 않고, classifier에서 나온 gradient 사용)에 "Pembroke Welsh corgi(웰시코기)"를 classifier에서 예측했을 때 샘플링한 결과.
- 왼쪽은 classifier scale로 1.0을 사용하였고 오른쪽은 10.0을 사용하였다. **FID는 왼쪽이 33.0, 오른쪽이 12.0**으로 클래스에 더 일치하는 이미지가 생성.

Classifier Guidance

Scaling Classifier Gradients

$$s \cdot \nabla_x \log p(y|x) = \nabla_x \log \frac{1}{Z} p(y|x)^s,$$

- Z: 임의의 상수
- s: scaling factor



- $s > 1$ 일 때 지수에 의해 값이 증폭되기 때문에 $p(y|x)^s$ 는 $p(y|x)$ 보다 더 뾰족해진다.
- 더 큰 scale을 사용하면 classifier의 모드(**classifier가 예측하는 클래스들**)들에 더 초점을 맞추며, 높은 fidelity면서 낮은 diversity의 샘플을 생성.
- 기본 확산 모델이 $p(x)$ 를 모델링하고 unconditional이라고 가정. 같은 방식으로 conditional diffusion model $p(x|y)$ (클래스를 미리 지정)를 학습시키고 classifier guidance를 사용 가능.

Classifier Guidance

Scaling Classifier Gradients

Conditional	Guidance	Scale	FID	sFID	IS	Precision	Recall
$\times$	$\times$		26.21	6.35	39.70	0.61	0.63
$\times$	✓	1.0	33.03	6.99	32.92	0.56	0.65
$\times$	✓	10.0	12.00	10.40	95.41	0.76	0.44
✓	$\times$		10.94	6.02	100.98	0.69	0.63
✓	✓	1.0	4.59	5.25	186.70	0.82	0.52
✓	✓	10.0	9.11	10.93	283.92	0.88	0.32

- **Conditional:** 직접 이미지에 대한 class를 $p(x|y)$ 의 형태로 넣어주는 방식 (ex. class y가 강아지일 때의 이미지 x 값)
- **Guidance:** 학습한 classifier에서 나온 gradient를 이용한 Class guidance 방식 (실제 이미지에 대한 class 이용 x)
- Scale factor가 크게 주어진다면, guidance만 했을 때가 conditional를 했을 때 만큼 FID가 비슷하다.
- Precision 과 Recall은 반비례하는 모습을 보여준다. (fidelity vs diversity)

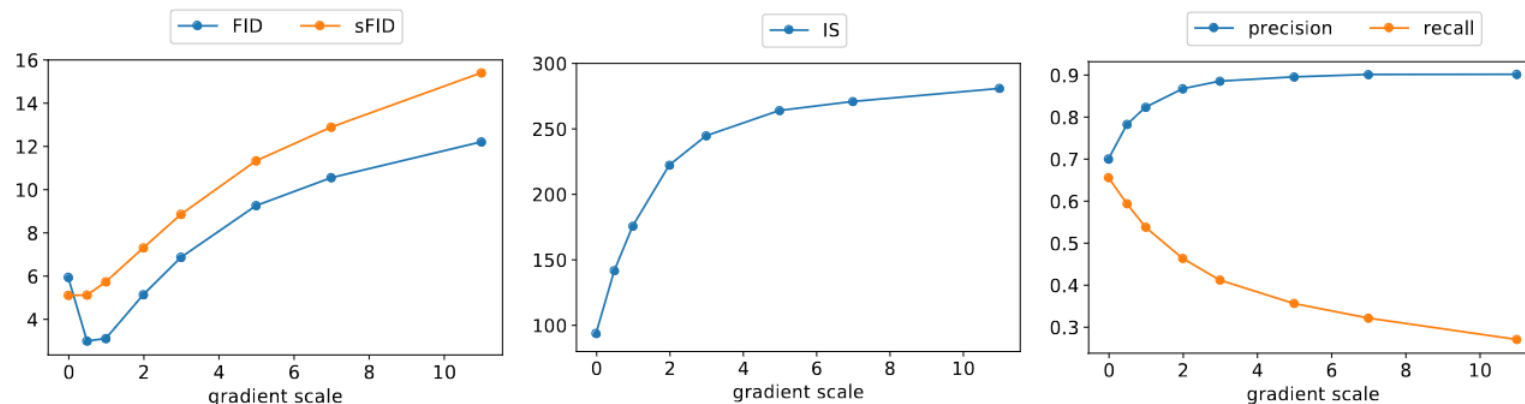
➔ Guide된 unconditional model이 guide되지 않은 conditional model의 FID에 상당히 근접할 수 있음.

➔ 물론 conditional model(직접 클래스 레이블을 사용)을 guide하면 FID가 더욱 향상됨.

Classifier Guidance

Scaling Classifier Gradients

- Gradient scale에 따른 척도 (FID, sFID, IS, precision, recall) 비교



- 두번째 그래프:** 1.0 이상의 scale을 사용하면 recall (diversity의 척도)을 희생해서 점점 높은 precision과 IS (fidelity의 척도)를 얻을 수 있음.
- 첫번째 그래프:** FID와 sFID는 fidelity와 precision을 둘 다 고려하는 척도라 중간 어딘가 에서 가장 낮은 값을 가짐.

Classifier Guidance

Scaling Classifier Gradients

- BigGAN의 truncation trick과 classifier guidance를 비교

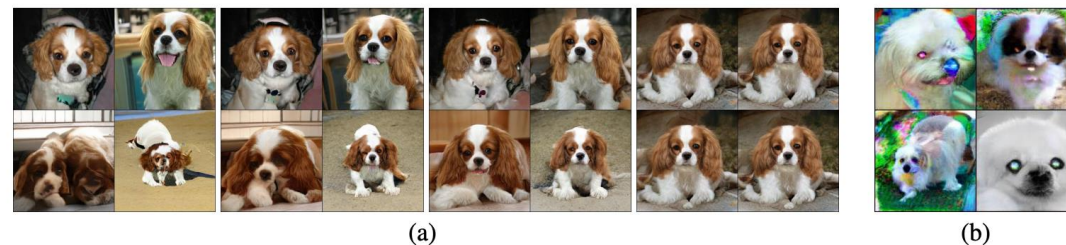
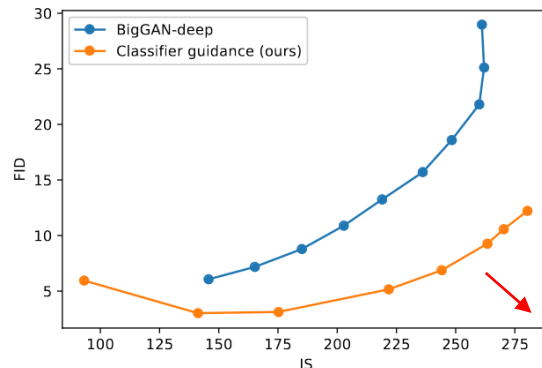
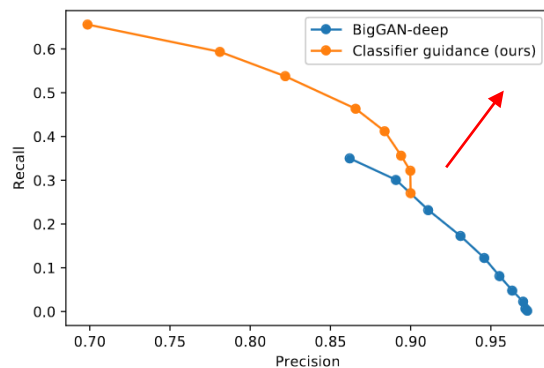


Figure 2: (a) The effects of increasing truncation. From left to right, the threshold is set to 2, 1, 0.5, 0.04. (b) Saturation artifacts from applying truncation to a poorly conditioned model.

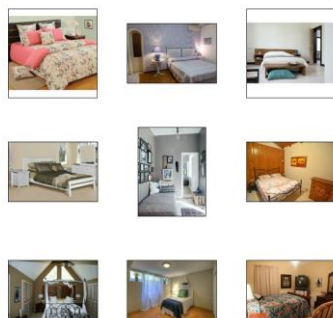
- *** truncation trick: GAN의 generator 모델에서 classifier guidance와 비슷하게 diversity와 fidelity를 조절하는 방법.
- train과정에서는 latent space인 z 를 normal distribution을 그대로 사용. evaluate시 z 값이 threshold 값을 넘어가게 되면 다시 추출하는 방법. Threshold 값이 작아질 수록 이미지 다양성은 떨어지지만 품질이 올라감.
- Precision/recall 그래프서 classifier guidance가 특정 정밀도 구간을 제외하고는 낮지 않았음 (왼쪽).
- Inception Score/FID에서 classifier guidance가 BigGAN-deep보다 훨씬 나은 성능을 보임. (오른쪽)



Results

State-of-the-art Image Synthesis

- LSUN(침실, 말, 고양이) 데이터셋 및 ImageNet 데이터셋(64x64, 128x128, 512x512)의 다양한 해상도로 테스트 진행



Model	FID	sFID	Prec	Rec
LSUN Bedrooms 256×256				
DCTransformer [†] [42]	6.40	6.66	0.44	0.56
DDPM [25]	4.89	9.07	0.60	0.45
IDDPM [43]	4.24	8.21	0.62	0.46
StyleGAN [27]	2.35	6.62	0.59	0.48
ADM (dropout)	1.90	5.59	0.66	0.51

LSUN Horses 256×256				
StyleGAN2 [28]	3.84	6.46	0.63	0.48
ADM	2.95	5.94	0.69	0.55
ADM (dropout)	2.57	6.81	0.71	0.55

LSUN Cats 256×256				
DDPM [25]	17.1	12.4	0.53	0.48
StyleGAN2 [28]	7.25	6.33	0.58	0.43
ADM (dropout)	5.57	6.69	0.63	0.52

ImageNet 64×64				
BigGAN-deep* [5]	4.06	3.96	0.79	0.48
IDDPM [43]	2.92	3.79	0.74	0.62
ADM	2.61	3.77	0.73	0.63
ADM (dropout)	2.07	4.29	0.74	0.63

Model	FID	sFID	Prec	Rec
ImageNet 128×128				
BigGAN-deep [5]	6.02	7.18	0.86	0.35
LOGAN [†] [68]	3.36			
ADM	5.91	5.09	0.70	0.65
ADM-G (25 steps)	5.98	7.04	0.78	0.51
ADM-G	2.97	5.09	0.78	0.59

ImageNet 256×256				
DCTransformer [†] [42]	36.51	8.24	0.36	0.67
VQ-VAE-2 ^{†‡} [51]	31.11	17.38	0.36	0.57
IDDPM [‡] [43]	12.26	5.42	0.70	0.62
SR3 ^{†‡} [53]	11.30			
BigGAN-deep [5]	6.95	7.36	0.87	0.28
ADM	10.94	6.02	0.69	0.63
ADM-G (25 steps)	5.44	5.32	0.81	0.49
ADM-G	4.59	5.25	0.82	0.52

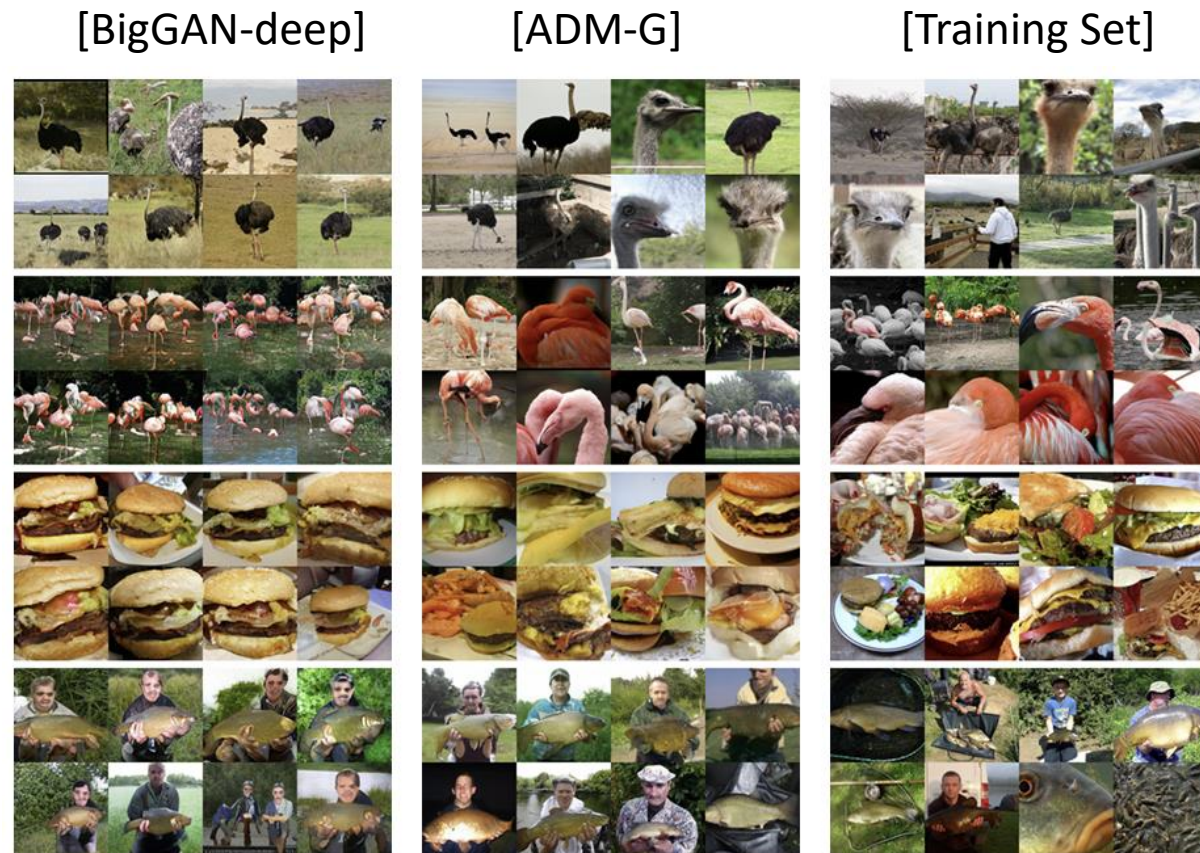
ImageNet 512×512				
BigGAN-deep [5]	8.43	8.13	0.88	0.29
ADM	23.24	10.19	0.73	0.60
ADM-G (25 steps)	8.41	9.67	0.83	0.47
ADM-G	7.72	6.57	0.87	0.42

- ADM : Ablated Diffusion Model 이라고 표기
- ADM-G : Classifier guidance 사용
- LSUN 모델은 1000 step으로 샘플링되었으며, ImageNet 모델은 250 step으로 샘플링되었음
- 대부분의 경우 ADM은 다양한 데이터셋 및 해상도에서 우수한 성능을 보여주는 것으로 보이며, 1개의 Task (LSUN Cats 256x256)를 제외한 나머지 Task에서 FID 및 sFID 점수가 가장 좋음
- 더 높은 해상도(128x128 이상) 의 ImageNet 데이터에 대해서는 classifier guidance를 통해 GAN 보다 높은 품질의 이미지를 생성

Results

State-of-the-art Image Synthesis

- 가장 성능이 좋은 BigGAN-deep 모델과 가장 성능이 좋은 ADM-G와 임의의 샘플을 비교.
- 이미지의 quality 는 비슷하지만 ADM이 더 다양한 이미지를 산출.



Results

Comparison to Upsampling

- Classifier guidance 의 결과를 two-stage upsampling stack 결과와 비교
- **Two Stage upsampling**: Low-resolution model 은 낮은 화질의 이미지를 생성하고, upsampling model은 낮은 화질 이미지를 고화질의 이미지로 바꾼다. (upsampling 모델이름: SR3)
- 저화질의 이미지를 **condition**으로 이용 고화질 이미지 생성
- 이 기법을 통해 SR3 모델의 경우, ImageNet 256×256 image 에 대한 FID score 를 개선시켰지만, BigGAN-deep 만큼의 quality 를 얻지는 못하였음

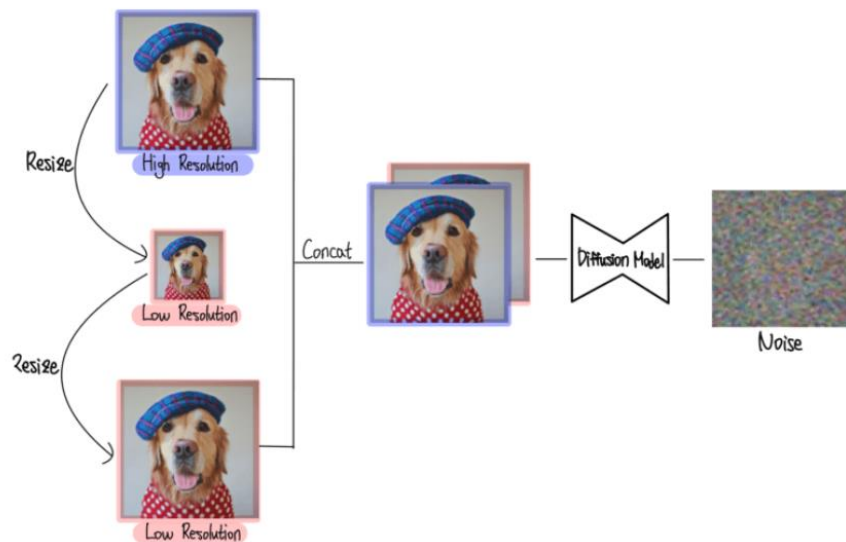


그림3. SR3 Super-Resolution 방법

Model	FID	sFID	Prec	Rec
ImageNet 256×256				
DCTransformer [†] [42]	36.51	8.24	0.36	0.67
VQ-VAE-2 ^{†‡} [51]	31.11	17.38	0.36	0.57
IDDP [‡] [43]	12.26	5.42	0.70	0.62
SR3 ^{†‡} [53]	11.30			
BigGAN-deep [5]	6.95	7.36	0.87	0.28
ADM	10.94	6.02	0.69	0.63
ADM-G (25 steps)	5.44	5.32	0.81	0.49
ADM-G	4.59	5.25	0.82	0.52

Results

Comparison to Upsampling

Model	S_{base}	$S_{upsample}$	FID	sFID	IS	Precision	Recall
ImageNet 256×256							
ADM	250		10.94	6.02	100.98	0.69	0.63
ADM-U	250	250	7.49	5.13	127.49	0.72	0.63
ADM-G	250		4.59	5.25	186.70	0.82	0.52
ADM-G, ADM-U	250	250	3.94	6.14	215.84	0.83	0.53
ImageNet 512×512							
ADM	250		23.24	10.19	58.06	0.73	0.60
ADM-U	250	250	9.96	5.62	121.78	0.75	0.64
ADM-G	250		7.72	6.57	172.71	0.87	0.42
ADM-G, ADM-U	25	25	5.96	12.10	187.87	0.81	0.54
ADM-G, ADM-U	250	25	4.11	9.57	219.29	0.83	0.55
ADM-G, ADM-U	250	250	3.85	5.86	221.72	0.84	0.53

- Upsampling model은 Improved DDPM의 **upsampling stack**을 ADM에 적용한 것으로, **ADM-U**라 표기
 - Guidance와 Upsampling이 **서로 다른 방향으로** 샘플의 품질을 향상
 - Upsampling은 recall을 높게 유지한 채로 precision을 개선, guidance는 더 높은 precision을 위해 diversity(낮은 recall)를 절충.
 - Low-resolution model 에 guidance 를 적용시키고 upsampling model 을 통과시켜, 가장 좋은 FID score (3.94, 3.85) 를 얻음.
- > Upsampling과 Class Guidance 두 방법은 **서로를 보완하는 관계**이다.

Limitations and Future Work

- 여러 개의 denoising step을 사용하기 때문에 샘플링 시간에서 여전히 GAN보다 느림
- Single step model의 샘플은 아직 GAN과 경쟁력이 없지만 이전의 single-step likelihood-based model보다 훨씬 나음
- 제안된 classifier guidance는 레이블이 있는 데이터셋에서만 사용할 수 있으며, 레이블이 없는 데이터셋의 fidelity를 위해 diversity를 교환하는 효과적인 전략은 아직 없음
- 향후에는 **unlabeled data**로 method 확장 가능
- (1) sample clustering을 통해 synthetic label 생성,
- (2) discriminative model training하여 샘플이 실제 데이터 분포에 있는지 예측

논문정리

Ablated Diffusion Model (ADM)은

아키텍처를 개선(Attention head를 늘리고, 여러 resolution attention 사용 등)하고,

Classifier Guidance (Training Set의 라벨을 학습)를 통해서

GAN의 이미지 생성 성능을 뛰어넘었고 특히 **고화질의 이미지** 생성에서도 뛰어난 성능을 보였다.

Thank You!

!